

Cybersecurity SLM Architecture & RAG Pipeline

Comprehensive Codebase Catalog, File Functionality & Disk Space Analysis

Root Directory: D:\ARBOOD\ARCHITECHTURE | Total Workspace Footprint: 7.33 GB

1. Executive Summary & Storage Distribution

This document provides a complete technical audit of the Cybersecurity SLM Architecture repository. The workspace currently occupies 7.33 GB (7,871,219,098 bytes). Crucially, 97.4% (7.14 GB) consists of the vector database engine, semantic vector embeddings, and compiled RAG security datasets. An additional 2.5% (188.7 MB) represents the standalone Qdrant binary distribution. The core application source code, operational automation scripts, specialist agent harnesses, and documentation are exceptionally lightweight, consuming under 5 MB combined.

TOTAL SIZE 7.33 GB All files & DBs	VECTOR DB ENGINE 3.89 GB Active Qdrant storage	RAG STAGING FILES 3.23 GB Corpus JSONL & SQLite	CODE & SCRIPTS < 5.0 MB Core Python & tools
---	---	--	---

High-Level Directory Size Breakdown

Directory	Size (GB)	Size (MB)	% Total	Operational Role & Contents
rag/	7.14 GB	7,314.9 MB	97.4%	Live Qdrant database storage and compiled RAG security datasets
tools/	0.18 GB	188.7 MB	2.5%	Standalone Qdrant server binary (qdrant.exe) & security tool manifests
data/	0.002 GB	2.1 MB	< 0.1%	Continuous learning, reflection, and feedback SQLite database
src/	< 0.001 GB	0.4 MB	< 0.1%	Core CyberAgent engine (orchestrator, routing, memory adapters)
scripts/	< 0.001 GB	0.3 MB	< 0.1%	Automation scripts (RAG search, ingestion, model helpers, tests)
tests/	< 0.001 GB	0.1 MB	< 0.1%	Pytest test suite (35 automated unit/integration tests) and fixtures
docs/	< 0.001 GB	0.1 MB	< 0.1%	System architecture specifications, workflow HTML, and runbooks
config/ & prompts/	< 0.001 GB	0.05 MB	< 0.1%	Routing rules, model endpoints specs, policy YAML, and agent prompts

2. RAG & Vector Database Storage (rag/ - 7.14 GB)

The rag/ directory is the primary data repository for retrieval-augmented generation. It contains the live Qdrant vector database storage engine (3.89 GB) housing 1,914,870 vectors, accompanied by the compiled text JSONL files and deduplication SQLite ledgers used during bulk ingestion.

File / Folder Name	Size	Classification	Technical Contents & Operational Work
rag/qdrant_server/	3.89 GB	Active Database Engine	Physical storage for 1.91M points across 3 collections (vectors, mmap, WAL, payloads).
rag/corpus.jsonl	1.02 GB	Compiled Knowledge Feed	Extracted, deduplicated text lines of NVD CVEs, CISA KEV, MITRE ATT&CK, CWE, CAP, etc.
rag/code_corpus.jsonl	940.7 MB	Compiled Code Feed	Vulnerable & secure code snippets from Juliet suite, CodeQL rules, Bandit, Gosec, Gitleaks, etc.
rag/corpus_core.jsonl	907.1 MB	Historical Staging Feed	Initial bootstrap subset of core cybersecurity vulnerability records.
rag/import.sqlite3	179.1 MB	Deduplication Ledger	SQLite database recording SHA256 content hashes and chunk IDs to prevent duplicate ingestion.
rag/import_core.sqlite3	124.6 MB	Core Deduplication State	Hash tracking ledger for the core knowledge corpus.
rag/qdrant/	104.2 MB	Legacy Embedded Store	Local SQLite-backed Qdrant store created during prototype testing before deploying the server.
rag/code_import.sqlite3	31.4 MB	Code Deduplication State	Content hash ledger ensuring unique code samples in the code review collection.
rag/ingest_state.sqlite3	4.8 MB	Resumption Checkpoint	Batch cursors and streaming state allowing interrupted ingest jobs to resume cleanly.

LIVE QDRANT VECTOR COLLECTIONS CURRENTLY ACTIVE IN rag/qdrant_server/:
1. cyberagent_memory : 1,215,043 points (128d, Cosine) | NVD CVEs, CISA KEV, MITRE ATT&CK, NIST SP 800-218
2. cyberagent_code : 689,939 points (128d, Cosine) | Juliet C++/Java, OWASP Juice Shop, CodeQL, Bandit, Gitleaks
3. cyber_semantic_rag: 9,888 points (384d, Cosine) | High-precision FastEmbed dense vectors for semantic search

3. Tools, Binaries & Manifests (tools/ - 188.7 MB)

The tools/ directory packages the pre-compiled database server binaries and declarative capability manifests used by specialist agents. The active executable is tools/qdrant/qdrant.exe (v1.13.2).

File / Artifact Path	Size	Operational Purpose & Work Performed
tools/qdrant/qdrant.exe	80.8 MB	Active Windows x86_64 vector DB server. Spawned by start_qdrant.ps1.
tools/qdrant/extract/qdrant.exe	80.8 MB	Duplicate uncompressed binary copy from initial release extraction (safe to remove).
tools/qdrant/qdrant.zip	27.1 MB	Original zip archive of the Qdrant release package (safe to remove).
tools/blue_team/manifest.yaml	< 1 KB	Declarative schema for SOC alert triage, SIEM log analysis, and IOC extraction.
tools/code_review/manifest.yaml	< 1 KB	Tool definition for AST-based SAST scanning, taint analysis, and remediation diffs.
tools/red_team/manifest.yaml	< 1 KB	Tool schema for attack surface mapping and safe vulnerability reproduction.
tools/cyber_generalist/manifest.yaml	< 1 KB	General vulnerability research and MITRE ATT&CK / CWE taxonomy alignment tools.
tools/common/manifest.yaml	< 1 KB	Cross-cutting tools: hybrid vector search, knowledge retrieval, and web search.

4. Core CyberAgent Engine (src/cyberagent/ - 0.4 MB)

The src/cyberagent/ package contains the entire production orchestration engine. It implements request routing, multi-provider model gateways, safety validation, and modular memory adapters.

Module / Component	Subsystem	Technical Work & Architectural Role
orchestrator.py	Control Plane	Coordinates end-to-end request lifecycle: routing -> context retrieval -> harness -> audit logging.
routing.py	Intent Routing	Classifies user queries against security heuristics and routes to the best specialist agent.
runtime.py	Service Container	Manages ContextBroker, ModelRegistry, and database connections across the app lifecycle.
model_gateway.py	Inference Layer	Unified async client supporting local Ollama, llama-server (GGUF), and remote RunPod endpoints.
contracts.py	Domain Models	Strict Pydantic v2 schemas for chat requests/responses, context snippets, and tool calls.
config.py	Configuration	Centralized environment loader reading .env with automatic type validation.
validation.py	Safety Guardrails	Enforces security guardrails, red-team boundaries, and output format compliance.
evaluation.py	Benchmarking	Automated evaluation engine computing accuracy, harness coverage, and pass rates.
main.py / cli.py	Entrypoints	FastAPI web application factory (REST, SSE streaming, WebSocket) and CLI commands.
memory/embeddings.py	Storage Adapter	Embedding providers: FastEmbed ONNX (384d), Hash deterministic (128d), and OpenAI API.
memory/vector_stores.py	Storage Adapter	Vector store implementations: high-throughput QdrantVectorStore and InMemoryVectorStore.
memory/graph_stores.py	Storage Adapter	Knowledge graph adapters: Graphiti/Neo4j entity relations and InMemoryGraphStore.
memory/audit_stores.py	Storage Adapter	Compliance persistence: PostgreSQL asyncpg store and in-memory audit logs.
memory/stores.py	Storage Adapter	Backward-compatibility re-export shim preserving existing import paths.
harness/*.py	Specialist Harness	Execution harnesses for Blue Team (triage), Red Team (emulation), Code Review, Generalist.

5. Operational Automation Scripts (scripts/ - 0.3 MB)

The scripts/ directory houses classified operational tools for RAG vector search, dataset ingestion, local and cloud model management, automated smoke tests, and specialist quality benchmarks. Dual invocation is supported: scripts can be run from their categorized subfolder or via root shims.

Script Path	Category	Operational Work & Invocation Command
scripts/rag/query_rag.py	RAG Retrieval	Semantic search & metadata filtering: python scripts/query_rag.py -q "..." -k 5
scripts/rag/manage_qdrant.py	RAG CRUD	Database maintenance, statistics, and CRUD: python scripts/manage_qdrant.py stats
scripts/rag/ingest_qdrant.py	RAG Ingestion	High-throughput dataset indexer: python scripts/rag/ingest_qdrant.py --preset curated
scripts/rag/build_local_qdrant.py	RAG Bulk Load	Bulk JSONL point upsert utility bypassing HNSW during initial ingestion.
scripts/rag/import_datasets_rag.py	RAG Compiler	Streaming parser compiling raw datasets in D:\ARBOOD\DATASETS into corpus.jsonl.
scripts/start_qdrant.ps1	RAG Daemon	PowerShell daemon starter launching Qdrant Windows server in background.
scripts/models/ollama_helper.py	Local Models	Checks local Ollama service, pulls pilot models, and tests inference completions.
scripts/models/llamacpp_helper.py	Local Models	Verifies llama-server GGUF inference endpoints and latency.
scripts/models/runpod_preflight.py	Cloud Preflight	Checks connectivity and authentication for remote RunPod GPU endpoints.
scripts/testing/smoke_test.py	Verification	Full end-to-end orchestration smoke test across all 5 specialist routing paths.
scripts/testing/api_smoke_test.py	Verification	Validates FastAPI HTTP endpoints, SSE streaming responses, and WebSockets.
scripts/testing/evaluate_harness.py	Evaluation	Runs deterministic evaluation suite from config/evaluations.yaml.
scripts/benchmarks/slm_harness_benchmark.py	Benchmarks	Evaluates 3-tier benchmark (Raw vs Script vs Harness) across 4 SLM roles.

6. Configuration, Prompts, Tests & Runtime State

File / Folder Path	Size	Purpose & Content Description
config/routing_rules.yaml	2.1 KB	Heuristic keywords and regex patterns routing queries to specialist SLM agents.
config/models.yaml	1.8 KB	Endpoint specs, context window limits, and parameter defaults for models.
config/policy.yaml	1.4 KB	Security guardrails, prohibited action constraints, and safety filters.
config/evaluations.yaml	3.2 KB	Evaluation dataset containing benchmark security prompts and expected outputs.
prompts/*.md (5 files)	14.5 KB	System persona prompts for Blue Team, Red Team, Code Review, Generalist, Utility.
data/learning.sqlite3	2.1 MB	Continuous learning and reflection SQLite database storing agent interaction traces.
tests/*.py (5 files)	29.9 KB	Pytest automated test suite covering gateways, harnesses, learning, and policies (35 tests).
tests/conftest.py	0.7 KB	Shared pytest fixtures providing mock_settings and mock_runtime lifecycle managers.
docs/ARCHITECTURE.md	2.8 KB	Comprehensive architectural map detailing directory structures and design patterns.
docs/architecture_workflow.html	19.1 KB	Interactive SVG/HTML visualization diagram of the complete SLM workflow.

! 7. Actionable Disk Space Optimization Guide

The workspace currently occupies 7.33 GB. If you wish to optimize disk storage, up to 3.44 GB (46.9% of total repository size) can be safely reclaimed without breaking the application, the vector database, or live RAG search capabilities.

Candidate File for Cleanup	Space Saved	Safety Assessment & Technical Rationale
rag/corpus.jsonl	1,040.0 MB	100% SAFE. All records are permanently stored in Qdrant; raw data exists in DATASETS.
rag/code_corpus.jsonl	940.7 MB	100% SAFE. 689,939 code chunks are already indexed inside cyberagent_code collection.
rag/corpus_core.jsonl	907.1 MB	100% SAFE. Redundant early-stage subset superseded by corpus.jsonl and Qdrant DB.
rag/import.sqlite3	179.1 MB	SAFE. Deduplication hash ledger for corpus.jsonl; only needed when re-ingesting.
rag/import_core.sqlite3	124.6 MB	SAFE. Redundant historical deduplication tracking ledger.
rag/code_import.sqlite3	31.4 MB	SAFE. Hash tracking ledger for code dataset import; already loaded into Qdrant.
rag/qdrant/ (legacy dir)	104.2 MB	SAFE. Obsolete embedded storage replaced by the active standalone qdrant_server/.
tools/qdrant/extract/qdrant.exe	80.8 MB	100% SAFE. Exact duplicate of tools/qdrant/qdrant.exe.
tools/qdrant/qdrant.zip	27.1 MB	100% SAFE. Download zip archive; binary is already extracted and operational.

SUMMARY OF POTENTIAL DISK SPACE RECOVERY:

* Intermediate RAG JSONL & SQLite Staging Files: ~3,227 MB (~3.23 GB)

* Redundant Legacy Qdrant Storage & Archives: ~212 MB (~0.21 GB)

= Total Space Reclaimable: ~3.44 GB (Reduces repository footprint from 7.33 GB to ~3.89 GB)

Note: The active vector database in rag/qdrant_server/ (3.89 GB) will remain 100% operational and intact.